

Final Exam Information

COP3502

Spring 2026

Time and Location

- 1:00 PM!!! (arrive before 1:00 PM)
- Common CS1 Final Exam date: Saturday, May 2nd, 2026, The exam will start at 1:00 PM
- It will be 2 hours and 30 minutes written exam
- Location: Refer to Webcourses (Do not take exam in any other room with other sections; otherwise, your exam paper will be misplaced)
- Please bring your UCF ID or your driver's license (if you don't have a UCF ID) with you to check in.

Written Exam Environment

- Fully written exam
- Question paper will be provided
- There will be space to answer the questions
- Questions should be answered in the question paper
- **NO calculator and any other electronic devices!**
- Total point: 100 (it will cover 25% of your course grade)

Topics and point distribution

- Topics from midterm ~20-23 pts and Topics from the final part (starting from summation to the end) ~76-80 pts
- A. Multiple Choice/Short Answer/Fill In The Blanks – 5%
- B. Summation – 7%
- C. Recurrence Relation – 7%
- D. Sorting Algorithms – 12%
- E. Binary Trees – 10%
- F. Binary Heaps – 10%
- G. Tries – 8%
- H. Bitwise Operators and Base Conversion – 7%
- I. Hash Tables – 6%
- J. AVL Trees – 8%
- K. Dynamic Memory Allocation – 5%
- L. Linked Lists – 5%
- M. Stack/Queue – 5%
- N. Algorithm Analysis (minus Sums/Recurrence) – 5%

Topics for Final Exam

1. Summation and Algorithm Analysis 3: (7 pts)

- Deriving summation from a code and perform run-time analysis
- Solving a given summation
- Related notes and examples from the lecture
- exercises and extra exercise/example pdf available on webcourses

2. Recurrence relation and related items (7 pts):

- Code to recurrence and solving it,
- Solve a given recurrence relation,
- related notes and examples from lecture
- exercises, and extra examples pdf examples, etc.

Topics for Final Exam

3. Sorting (12 pts):

For each sorting algorithm, you must know:

- Algorithm
- simulation steps (like exercises),
- run-time analysis (With steps)
- Best/worst/average case of run-time, space complexity, and stability
- Code (know all the steps so that you can find bugs, modify, optimize, etc)
- When the best and worst case can happen
- Comparing multiple sorting algorithms
- Related notes,
- Extra example pdf available on webcours
- N-squared sorts:
 - selection,
 - insertion,
 - bubble
- Quick sort
- Merge sort

Topics for Final Exam

4. Binary trees (10 pts)

- related definitions and characteristics, node structure,
- traversal in order, preorder, post order, related algorithms, codes
- Binary search tree
 - related definitions and characteristics,
 - insertion,
 - deletion,
 - search,
 - various utility functions,
 - traversal inorder, preorder, post order, related algorithms,
 - run-time analysis,
 - codes , related notes, exercises,
 - Extra example pdf available on webcouses

Topics for Final Exam

5. Trie (8 pts):

- Motivation behind learning Trie and the disadvantages of using BST for strings
- Definition
- Insertion
- Deletion
- Run-times best/worst
- All the related coding including PrintAll coding
- Related notes
- Exercises,
- Extra example pdf

Topics for Final Exam

6. Bitwise operator and Base Conversion (7 pts):

- Base conversion and related shortcut methods
- Various bitwise operations (&, |, ~, ^, <<, >>) and use cases
- bit-masking,
- Two's complement (know that what is $\sim x$?)
- shifting and application,
- related codes,
- Iterating through subset of an array example
- Extracting, setting different bits of a number to solve a problem
- related notes
- exercises, and extra examples provided

Topics for Final Exam

7. Hash Table (6 pts):

- Purpose of Hash table
- Definitions,
- Features and problems, and operations
- Properties of a good hash function
- Linear probing
- Quadratic probing
- Separate Chaining hasing
- Related codes
- Related notes,
- Run-time
- Exercises,
- Extra example pdf uploaded on webcourses

Topics for Final Exam

8. Heap (10 pts)

- Related definitions and characteristics and use of heap,
- Heap and shape properties
- Insertion and coding
- Deletion and coding
- Percolateup and coding
- Percolatedown and coding
- Heapify and coding (building or constructing a heap)
- Heapsort and coding
- Run-time analysis,
- Relevant codes
- related notes,
- exercises pdf uploaded on webcourses

Topics for Final Exam

9. AVL Tree (8 pts):

- Problems with regular BST
- definitions,
- purpose,
- steps of various operations,
- insertion,
- deletion,
- balance factor,
- height,
- labeling ABC,
- Re-balancing/rotation (3 types of imbalance (LL/RR/LR/RL and how many rotations needed for them)
- related notes and examples
- and exercises

Topics for Final Exam (from midterm exam topics)

10. Dynamic memory allocation (5 pts):

- Static vs Dynamic memory allocation, stack and heap space, problems with VLA, malloc, calloc, realloc, free, memory leak,
- DMA of array in a function and free,
- DMA struct in a function and free,
- DMA array of struct in a function and free,
- DMA array of struct pointers where each pointer is a DMA array or struct.
- DMA 2D array with variable length,
- DMA a struct that has DMA array in it,
- DMA array of strings and load it from a file, inputs/passed array of strings, etc.
- All the related exercises
- Make sure to understand the solution of PA1

Topics for Final Exam (from midterm exam topics)

11. Linked list (5 pts):

- Concept, array vs linked list
- singly linked list, doubly linked list, circular linked list concepts
- How to insert at the beginning, at the end, and in between nodes in a linked list
- How to delete an item by searching, or deleting an item from the beginning, from the end, in between nodes in a linked list, and freeing the entire list
- Sorted linked list coding
- Related codes, examples, exercises, output tracing, function writing,
- Tracing a linked list function to see the status of the list after the function
- Practice based on the exercises (do not skip this!)
- and Writing recursive code for various operations on a linked list

Topics for Final Exam (from midterm exam topics)

12. Stack/Queues (5 pts):

- Queues

- Overall Queue Concept (FIFO), applications of queues, and various queue functions (enqueue, dequeue, peek, full, empty), and
- linked list-based implementation and its efficiency gain by using the back pointer, and related coding
- Array-based implementation-linear queue and coding, and issues with linear queue
- Circular queue, enqueue, and dequeue steps and coding
- For any coding parts, make sure you know the purpose of each part of the code
- Practice based on the examples of the lab

- Stacks

- Overall Stack Concept (LIFO), application of stacks, and various stack functions (Push, pop, peek, full, empty)
- Linked list-based implementation and coding
- Array-based implementation and coding
- Another application could be palindrome checking using a stack (we have not done this in the class, But, you can build up the logic easily if you understand the push and pop and also other examples we have learned)
- Infix to postfix with steps
- Post-fix to evaluation with steps
- Do the exercises at the end of the slide
- Also, know how we have solved them in the class, which shows the status of the stack at various stages during the infix and postfix processes as well as during the evaluation process.

Topics for Final Exam (from midterm exam topics)

13. Algorithm Analysis (minus Sums/Recurrence) – 5pts

- SLMP Binary Search and Ideas about Run-time:
 - Concept of linear search and run-time, and coding
 - Concept and steps of binary search, iterative and recursive coding, And run-time analysis of binary search. How is it $O(\log_2 n)$?
 - SLMP 3 versions and coding, and how the run-time is changing at each version and coding with the related mindset
- Algorithm Analysis 1:
 - What is Big-O and its purpose
 - Finding the Big-O from a mathematical expression
 - Common Orders (constant, logarithmic, linear, polylog, quadratic, Exponential, factorial.)
 - Math related to algorithm analysis (see the last two examples and the uploaded exercises and solutions)
 - Some common run-time of the data structure and algorithms we have learned so far
- Algorithm Analysis 2:
 - Given a function or a piece of code, derive the big-o run-time for that function/code
 - We have gone through many of them during the lecture
 - Also, do not skip the exercise and solutions posted on webcourses in the Algorithm Analysis module

How to prepare

If you have prepared well for the quizzes, you have already prepared yourself. (You know best what have you skipped while preparing for the quizzes!)

- If you have done your programming and lab assignments (from scratch, not just modify after copy-paste), you should have a pretty good knowledge on those related topics in terms of coding.
- If you have done the exercises and additional examples properly and understand them clearly, and if you have confidence that you can do them without looking at the solution, you should have a good knowledge on the topics!
- Go through the slides, notes, and class work, and uploaded discussed codes to review the concepts. Remember, exercises are not the replacement of the examples of the lecture notes.
- Practice and go through the examples in the slides and see can you do them without looking at the solution
- Do you understand why the run time of this operation is that?
- Do you understand the reason behind the use of a particular data structure?
- Do you understand what is happening in a particular loop or in a recursion?
- Go through the uploaded exercises and additional examples. These are very helpful as part of preparation.
- Again, if there is something in the slide, try to do it your self.
- Try to understand, how we have mapped an algorithmic concept into code in our various data structures.

Do you need some practice Questions?

- You have a lot of practice questions in the modules and practice problems
- Practice solving the final exam for Fall 2025
- Also, try with some past foundation exam questions!

<https://feprep.net/problems>

<https://passthefoundationexam.vercel.app/categorymode>

Final Note

- Good luck and I wish you all the best 😊